PROYECTO CINE
CON SMALLTALK

INFORME

Metodologías de Programación II

Franco Fernando Martins Agustin Melgar Luciano

2025

ÍNDICE 1
DESCRIPCIÓN DEL PROBLEMA PLANTEADO Y MODELO DE NEGOCIO 3
PROPÓSITO DEL PROYECTO 3
MODELO DE NEGOCIO IMPLEMENTADO 3
ESPECIFICACIÓN E IMPLEMENTACIÓN DE LAS CLASES UTILIZADAS 4
CLASES PRINCIPALES 4
CLASES AUXILIARES 8
UML 10
IMPLEMENTACIÓN DE LA APLICACIÓN DESARROLLADA 11
CARACTERÍSTICAS PRINCIPALES: 11
CLASES UTILIZADAS 11
MÉTODOS DESTACADOS DE LA APLICACIÓN: 12
CAPTURA DE LAS VISTAS DE LA APLICACIÓN: 15
DIAGRAMA DE SECUENCIA – REGISTRO DE FUNCIÓN Y ASIGNACIÓN DE ASIENTOS 20
METODOLOGÍA DE TRABAJO 21
ETAPAS DEL DESARROLLO: 21
ENFOQUES APLICADOS: 21
ROLES Y COLABORACIÓN 22
FRAMEWORKS ORIENTADOS A OBJETOS UTILIZADOS Y SUGERIDOS 23
FRAMEWORKS UTILIZADOS 23
FRAMEWORKS SUGERIDOS PARA FUTURAS EXPANSIONES 23
ALTERNATIVAS EN OTROS LENGUAJES ORIENTADOS A OBJETOS 24
IMPACTO DE LOS FRAMEWORKS EN EL PROYECTO 24
CASOS DE USO DE PATRONES EN EL CONTEXTO DEL PROBLEMA 25
PATRONES DE DISEÑO APLICADOS: 25
PATRONES SUJERIDOS: 27
REPOSITORIOS PARA DESARROLLO COLABORATIVO 28
REPOSITORIOS UTILIZADOS Y RECOMENDADOS 28
COMO SE APLICARON EN EL PROYECTO 28
IMPACTO DE LOS REPOSITORIOS EN EL PROYECTO 28
CONCLUSIÓN 29

Descripción del problema planteado y modelo de negocio

El sistema desarrollado tiene como objetivo modelar la lógica de funcionamiento de un **complejo de cines**, permitiendo gestionar de manera organizada distintos aspectos clave del negocio, como la administración de salas, películas, funciones, clientes y reservas. Se ha diseñado con una **estructura modular**, facilitando futuras expansiones, como la **gestión de empleados** y la incorporación de un **Candy bar**, sin afectar la estabilidad del sistema.

Propósito del proyecto

El sistema busca ofrecer:

- **Automatización de procesos administrativos**, reduciendo errores manuales en reservas y programación de funciones.
- **Gestión integral** de clientes, salas y funciones, asegurando una administración eficiente y escalable.
- **Interfaz intuitiva**, permitiendo una experiencia fluida para los administradores y clientes.
- **Expansión futura**, con un diseño adaptable que facilita la incorporación de nuevos módulos comerciales.

Modelo de negocio implementado

El cine opera sobre tres pilares fundamentales:

1. **Gestión de películas y funciones**
   - Registro de películas en cartelera con distintos formatos **(2D, 3D, 4D, IMAX).**
   - Programación de funciones con horarios específicos, asignados a una sala determinada.
   - Control de disponibilidad de asientos para cada función.
2. **Reservas y clientes**
   - Creación y gestión de clientes con identificación única.
   - Administración de reservas con selección de asientos y cálculo de costos.
   - Reportes sobre ocupación de salas y desempeño comercial.
3. **Administración de salas y estructura interna**
   - Manejo de salas, horarios y asientos de manera estructurada.
   - Gestión de datos eficiente, permitiendo futuras expansiones como **ventas en el Candy bar** y **administración de empleados**.
   - Implementación de una arquitectura orientada a objetos, utilizando **buenas prácticas de diseño y programación** para garantizar escalabilidad y mantenimiento del sistema.

Especificación e Implementación de las clases utilizadas

El sistema contiene múltiples clases organizadas para modelar el cine. Se incluyen:

- **Clases principales:** Cine, Cliente, Reserva, Pelicula, FuncionPelicula, Sala, CuentaCliente.
- **Clases auxiliares:** Horario, Asiento.

Cada clase gestiona atributos y métodos específicos, asegurando un diseño modular y reutilizable.

Clases Principales

Clase Cine

- **Función:** Clase principal del sistema que modela la administración del cine, gestionando la cartelera, salas, clientes y reservas.
- **Interacción con otras clases:** Se comunica con **Pelicula**, **Sala**, **Cliente** y **Reserva** para organizar los datos del cine.
- **Uso de colecciones:** Maneja un **Dictionary** para la cartelera, y un **OrderedCollection** para las salas y clientes.

Métodos Relevantes:

```
verPeliculasOrdenadas
	"mostrar en pantalla la lista de peliculas
	y retorna otra lista ordenada por nombre y luego por formato con strings (titulo+formato)
	indice titulo formato precio
	ej:
	1 Batman 3D $850
	2 Batman 4D $1000
	3 Spiderman2 2D $500
	4 Spiderman2 3D $850
	...
	"

	| listaPeliculasOrdenada lista p |
	lista := OrderedCollection new.
	listaPeliculasOrdenada := cartelera asSortedCollection:
	[:x :y |
	x verTitulo < y verTitulo or: [x verTitulo = y verTitulo and: [x verFormato < y verFormato]]].
	1 to: listaPeliculasOrdenada size
		do:
			[:i |
			p := OrderedCollection new.
			p add: (listaPeliculasOrdenada at: i) verTitulo.
			p add: (listaPeliculasOrdenada at: i) verFormato.
			lista add: p].
	^lista
```

Organiza las películas **por título y formato**, utilizando **Collect** para transformar la colección en una lista con información relevante.

```
nuevaFuncionPeliculaHorario: unaHora idioma: unIdioma formato: unFormato dia: unDia pelicula: unaPelicula sala: unaSala
    | funcion horario horaFin horas minutos |
    minutos := unaHora \\ 100 + unaPelicula verDuracion.
    horas := unaHora // 100 + (minutos // 60).
    horaFin := horas * 100 + (minutos \\ 60).
    horario := Horario
                crearHorarioDia: unDia
                Inicio: unaHora
                horaFin: horaFin
                nombrePelicula: unaPelicula verTitulo.
    (self agregarHorario: horario Sala: unaSala)
        ifTrue:
            [funcion := FuncionPelicula
                        crearFuncionPeliculaNombreSala: unaSala verNombreSala
                        nombrePelicula: unaPelicula verTitulo
                        horario: unaHora
                        idioma: unIdioma
                        formato: unFormato
                        dia: unDia.
            horario setIdFuncionPelicula: funcion verIdFuncionPelicula.
            1 to: (self seleccionarSala: unaSala verNombreSala) verAsientos size
                do:
                    [:i |
                    ((self seleccionarSala: unaSala verNombreSala) verAsientos at: i)
                        nuevoEstadoReserva: funcion verIdFuncionPelicula].
            ^self agregarFuncionPelicula: funcion pelicula: unaPelicula]
        ifFalse: [^false]
```

Este método **crea una instancia** de **FuncionPelicula**, asegurando que haya disponibilidad en el horario asignado. Utiliza otros **métodos refactorizados** de **Cine** para interactuar con **Pelicula**, **Sala** y **Horario**.

Interacción entre objetos

- **Horario:** Se crea una instancia para registrar el inicio y fin de la función, asignándole el nombre de la película.
- **FuncionPelicula:** Se instancia utilizando los datos de idioma, sala, formato y horario.
- **Sala:** Se accede a su colección de asientos, asegurando que cada uno actualice su disponibilidad para la nueva función.
- **Pelicula:** Se agrega la nueva función a su cartelera interna en Cine.

Clase Cliente

- **Hereda:** De la Clase **Persona** los atributos nombre, apellido, teléfono, dirección y email; con sus respectivos geters y seters .
- **Función:** Modela a los clientes del cine, almacenando datos personales y una instancia de **CuentaCliente**.
- **Interacción con otras clases:** Se comunica con **CuentaCliente** y **Reserva** para administrar las compras y reservas de funciones.

Método relevante:

```
verReservas
    "Devuelve las reservas realizadas por el cliente"

    ^self verCuenta verReservas select: [:r | r verCosto > 0]
```

Utiliza **select** para filtrar las reservas activas del cliente.

Clase Reserva

- **Función:** Representa las reservas realizadas por los clientes, incluyendo los asientos elegidos y el costo total.
- **Interacción con otras clases:** Se vincula a Cliente, **FuncionPelicula** y **Sala**, asegurando la disponibilidad de asientos.
- **Uso de colecciones:** Maneja un **OrderedCollection** con los asientos reservados.

Clase Pelicula

- **Función:** Administra la información de las películas, incluyendo título, duración, formato y funciones asociadas.
- **Interacción con otras clases:** Se comunica con **FuncionPelicula** para gestionar las funciones disponibles.
- **Uso de colecciones:** Usa un **Dictionary** para almacenar funciones de cada película.

Método relevante:

```
eliminarFuncionPelicula: idFuncionPelicula
    funciones removeKey: idFuncionPelicula ifAbsent: [^nil]
```

Este método **usa el diccionario** para eliminar la función de la cartelera si su clave (el Id de la función) coincide.

Clase FuncionPelicula

- **Función:** Representa una función específica de una película en una sala determinada, con atributos como horario, idioma y formato.
- **Interacción con otras clases:** Se relaciona con Sala, Horario, y Reserva para garantizar la programación correcta.

Clase Sala

- **Función:** Maneja la distribución de las salas dentro del cine, asegurando que las funciones se asignen correctamente.
- **Interacción con otras clases:** Trabaja con **Asiento**, **Horario**, y **FuncionPelicula** para la organización de las funciones de las películas.
- **Uso de colecciones:** Contiene un **OrderedCollection** con los asientos de la sala, y otro con los Horarios de las funciones en emisión.

Métodos relevantes:

```
agregarHorario: unHorario
    "Si agrega el elemento retorna correcto si no lo agrega retorna el elemento que encontro"

    | h |
    h := false.
    horarios detect: [:hor | (hor comprobarSupHoraria: unHorario) = true]
        ifNone:
            [horarios add: unHorario.
            h := true].
    ^h
```

Utilizando **detect** verifica que no exista superposición horaria con otra función en la sala antes de agregar un nuevo horario.

```
verAsientosDisp: idPelicula
    ^asientos select: [:a | (a comprobarReserva: idPelicula) = false]
```

Utiliza **select** para obtener los asientos libres de una función específica.

```
verAsientosRes: idPelicula
    ^asientos reject: [:a | (a comprobarReserva: idPelicula) = false]
```

Utiliza **reject** para obtener los asientos reservados de una función específica.

Clase CuentaCliente

- **Función:** Administra la cuenta de cada cliente, incluyendo credenciales y listado de reservas.
- **Interacción con otras clases:** Se conecta con **Cliente** y **Reserva** para gestionar la actividad del usuario.
- **Uso de colecciones:** Usa un **OrderedCollection** para almacenar las reservas del cliente.

Clases Auxiliares

Clase Horario

- **Función:** Define los horarios de las funciones de películas, asegurando que no haya superposiciones y permitiendo el control preciso de los tiempos.
- **Interacción con otras clases:** Se comunica con **FuncionPelicula** para asociar horarios a las funciones y con **Sala** para determinar la disponibilidad de horarios en cada sala.
- **Uso de colecciones:** No utiliza colecciones, pero se usa dentro de otras clases para validar superposiciones de horarios.

Métodos relevantes:

```
comprobarSupHoraria: unHorario
    ^unHorario verDiaHorario = dia and:
            [(unHorario verHoraInicio between: inicio and: fin)
                or: [unHorario verHoraFin between: inicio and: fin]]
```

Este método verifica si un **nuevo horario** entra en conflicto con otro ya existente en la misma sala, garantizando que no haya **superposiciones**.

- **Uso:** Se llama desde **Sala** cuando se intenta agregar un nuevo horario para verificar su disponibilidad.
- **Interacción:** Recibe una instancia de **Horario** y compara sus atributos **dia**, **inicio** y **fin** con los del objeto actual.

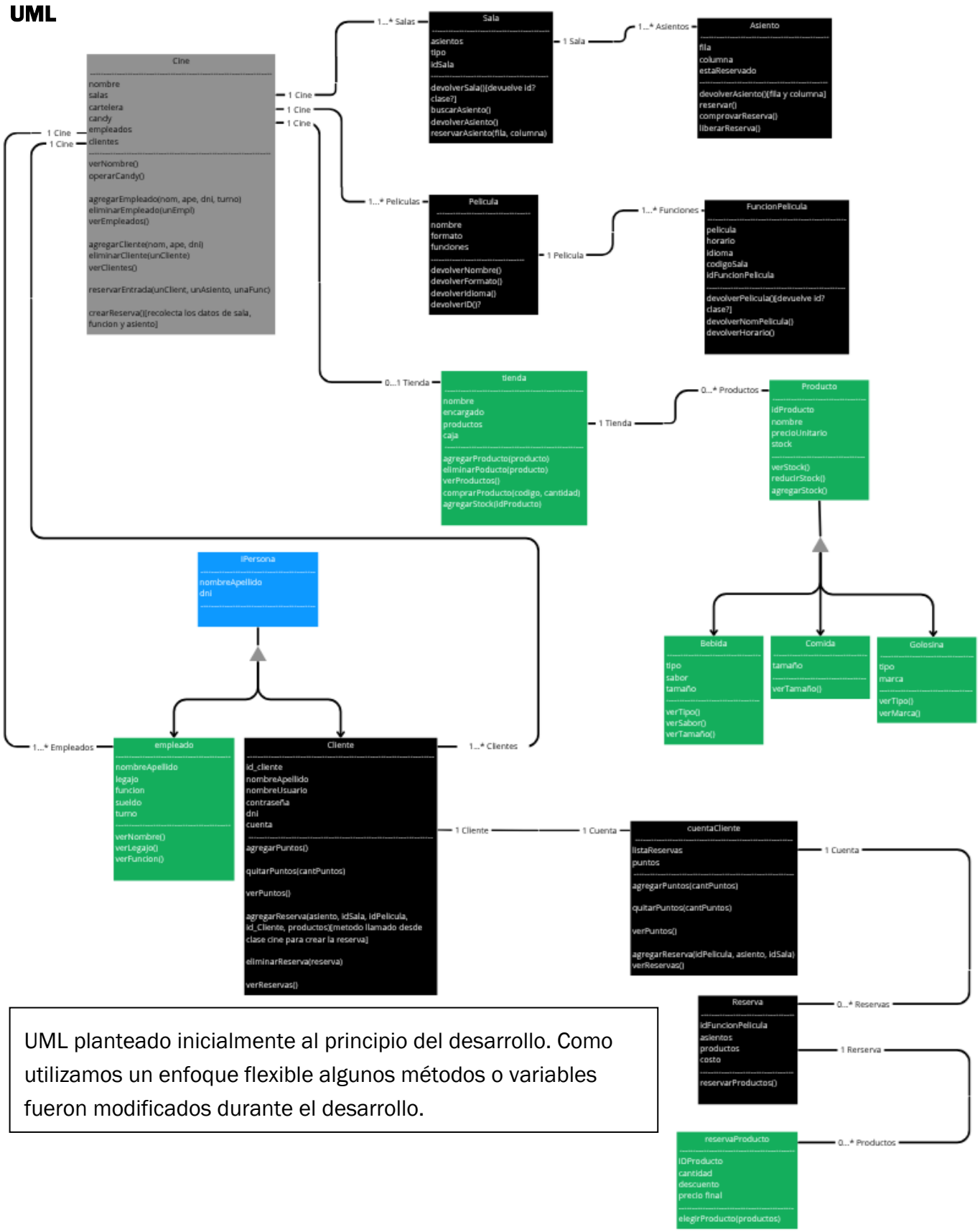
```
printOn: aStream
    | horaInicio horaFin minsInicio minsFin |
    horaInicio := self verHoraInicio // 100 >= 24
                ifTrue: ['0' , (self verHoraInicio // 100 - 24) printString]
                ifFalse:
                    [self verHoraInicio // 100 < 10
                        ifTrue: ['0' , (self verHoraInicio // 100) printString]
                        ifFalse: [(self verHoraInicio // 100) printString]].
    horaFin := self verHoraFin // 100 >= 24
                ifTrue: ['0' , (self verHoraFin // 100 - 24) printString]
                ifFalse:
                    [self verHoraFin // 100 < 10
                        ifTrue: ['0' , (self verHoraFin // 100) printString]
                        ifFalse: [(self verHoraFin // 100) printString]].
    minsInicio := self verHoraInicio \\ 100 < 10
                ifTrue: ['0' , (self verHoraInicio \\ 100) printString]
                ifFalse: [(self verHoraInicio \\ 100) printString].
    minsFin := self verHoraFin \\ 100 < 10
                ifTrue: ['0' , (self verHoraFin \\ 100) printString]
                ifFalse: [(self verHoraFin \\ 100) printString].
    aStream
        nextPutAll: 'Pelicula: ' , self verPeliculaEnEmision;
        nextPut: Character lf;
        nextPutAll: ' | Dia: ' , dia , ' | Hora de inicio: ' , horaInicio , ':' , minsInicio
                    , ' hs. - Hasta: ' , horaFin
                    , ':' , minsFin
                    , ' hs.'
```

Este método **da formato al horario** de una función para su visualización, asegurando que siempre se muestre en un formato claro.

Clase Asiento

- **Función:** Representa cada asiento de una sala, asegurando que pueda reservarse correctamente y evitando conflictos entre funciones.
- **Interacción con otras clases:** Se vincula a **Sala** y **Reserva**, administrando las ocupaciones de los asientos según la función seleccionada.
- **Uso de colecciones:** Usa un **Dictionary** (**estaReservado**) donde cada clave es el ID de una función y su valor es un **Boolean**, indicando si el asiento está reservado o no.

UML



UML planteado inicialmente al principio del desarrollo. Como utilizamos un enfoque flexible algunos métodos o variables fueron modificados durante el desarrollo.

Implementación de la aplicación desarrollada

La aplicación se desarrolló siguiendo un modelo **MVP** (**Model-View-Presenter**), asegurando una **separación clara** entre la lógica del negocio y la interfaz gráfica. Esto permite que las funcionalidades sean **modulares**, facilitando futuras expansiones.

Características principales:

1. **Gestión de funciones y películas:**
   - Creación de funciones asociadas a películas, con horarios, idiomas, y formatos (2D, 3D, 4D, IMAX).
   - Organización de la cartelera, permitiendo mostrar películas de manera ordenada.
2. **Administración de reservas y clientes:**
   - Registro de clientes con cuentas asociadas, asegurando un sistema de autenticación básico.
   - Creación de reservas, con selección de asientos en base a disponibilidad.
3. **Interfaz gráfica y visualización de datos:**
   - Implementación de pantalla principal (**CineShell**) con menús para gestionar películas, funciones, salas y clientes.
   - Módulos de información, permitiendo ver detalles en **InfoPelicula**, **InfoSala** y **InfoCliente**.
4. **Uso de métodos clave y colecciones:**
   - Se aplicaron métodos como **select**, **collect**, **reject**, y **detect** para manipular colecciones de objetos eficientemente.
   - Se utilizaron **OrderedCollection**, **SortedCollection** y **Dictionary** para almacenar datos en tiempo de ejecución.

Clases utilizadas

La aplicación cuenta con una interfaz basada en **MVP** utilizando las siguientes clases:

- **Modelos MVP:** CineModel, PeliculaModel, FuncionPeliculaModel, AsientoReservaModel, ReservaModel, SalaModel.
- **Interfaz (Presenters):** CineShell, CineNuevaPelicula, CineNuevaFuncionPelicula, CineNuevaReserva, CineNuevaSala, MenuSalas, InfoPelicula, InfoSala, InfoCliente.

Métodos destacados de la aplicación:

Clase CineModel:

```
buscarUnCliente: unString
    ^cine verClientes detect:
            [:cli |
            (cli verIdCliente printString findString: unString) > 0 or:
                    [(cli verDni printString findString: unString) > 0 or:
                            [(cli verNombre asLowercase findString: unString asLowercase) > 0
                                or: [(cli verApellido asLowercase findString: unString asLowercase) > 0]]]]
        ifNone: [^nil]
```

```
verPeliculasFiltradasBool2D: unBool2D bool3D: unBool3D bool4D: unBool4D boolIMAX: unBoolIMAX
    | cartelera |
    cartelera := unBool2D
                ifTrue: [self verPeliculas]
                ifFalse: [self verPeliculas reject: [:peli | peli verFormato = '2D']].
    cartelera := unBool3D
                ifTrue: [cartelera]
                ifFalse: [cartelera reject: [:peli | peli verFormato = '3D']].
    cartelera := unBool4D
                ifTrue: [cartelera]
                ifFalse: [cartelera reject: [:peli | peli verFormato = '4D']].
    cartelera := unBoolIMAX
                ifTrue: [cartelera]
                ifFalse: [cartelera reject: [:peli | peli verFormato = 'IMAX']].
    ^cartelera
```

```
verFuncionesFiltradasPelicula: unaPelicula idioma: unIdioma boolLunes: unBoolLunes boolMartes: unBoolMartes boolMiercoles: unBoolMiercoles boolJueves: unBoolJueves
boolViernes: unBoolViernes boolSabado: unBoolSabado boolDomingo: unBoolDomingo
    "Segun el idioma"
    | funcionesAMostrar |
    unIdioma = 'Todos'
        ifTrue: [funcionesAMostrar := self verFunciones: unaPelicula]
        ifFalse:
            [funcionesAMostrar := (self verFunciones: unaPelicula) select: [:fun | fun verIdioma = unIdioma]].
    "Segun los dias"
    unBoolLunes ifFalse: [funcionesAMostrar := funcionesAMostrar reject: [:fun | fun verDia = 'Lunes']].
    unBoolMartes
        ifFalse: [funcionesAMostrar := funcionesAMostrar reject: [:fun | fun verDia = 'Martes']].
    unBoolMiercoles
        ifFalse: [funcionesAMostrar := funcionesAMostrar reject: [:fun | fun verDia = 'Miércoles']].
    unBoolJueves
        ifFalse: [funcionesAMostrar := funcionesAMostrar reject: [:fun | fun verDia = 'Jueves']].
    unBoolViernes
        ifFalse: [funcionesAMostrar := funcionesAMostrar reject: [:fun | fun verDia = 'Viernes']].
    unBoolSabado
        ifFalse: [funcionesAMostrar := funcionesAMostrar reject: [:fun | fun verDia = 'Sábado']].
    unBoolDomingo
        ifFalse: [funcionesAMostrar := funcionesAMostrar reject: [:fun | fun verDia = 'Domingo']].
    ^funcionesAMostrar
```

```
verFunciones: unaPelicula
    | funciones dias |
    dias := Dictionary new.
    dias at: 'Lunes' put: 1.
    dias at: 'Martes' put: 2.
    dias at: 'Miércoles' put: 3.
    dias at: 'Jueves' put: 4.
    dias at: 'Viernes' put: 5.
    dias at: 'Sábado' put: 6.
    dias at: 'Domingo' put: 7.
    funciones := (cine verFuncionesDePelicula: unaPelicula) asSortedCollection:
                    [:x :y |
                    (x mayorTitulo: y) or: [(x mayorDia: y) or: [(x mayorHorario: y) or: [x mayorSala: y]]]].
    ^funciones
```

Clase ReservaModel:

```
cantAsientosSeleccionados
    asientos size > 0
        ifTrue: [^(asientos collect: [:asiento | asiento estado]) occurrencesOf: true]
        ifFalse: [^0]
```

Clase CineShell:

```
nuevaReserva
    | resModel nuevaReserva |
    resModel := ReservaModel new.
    resModel idFuncionPelicula: funciones selection verIdFuncionPelicula.
    (nuevaReserva := CineNuevaReserva showModalOn: resModel) notNil
        ifTrue:
            ["Filtra unicamente los asientos que fueron seleccionado (estado = true)"
            nuevaReserva asientos: (nuevaReserva asientos select: [:asi | asi estado]).
            "Verigica que se hayan ingresado los datos de id de la pelicula, se haya seleccionado un cliente y se haya selecionado al menos un asiento"
            (nuevaReserva idFuncionPelicula ~= ''
                and: [nuevaReserva idCliente > 0 and: [nuevaReserva asientos size > 0]])
                    ifTrue:
                        [(self model nuevaReserva: nuevaReserva)
                            ifTrue: [MessageBox warning: 'Reserva agregada exitosamente']
                            ifFalse: [MessageBox warning: 'No se ha podido realizar la reserva']]
                    ifFalse: [MessageBox warning: 'Los datos ingresados son incorrectos']]
```

```
eliminarPelicula
    | peliculaAeliminar confirmado funcionesPelicula |
    peliculaAeliminar := peliculas selection.
    funcionesPelicula := String streamContents:
                        [:s |
                        s
                            nextPut: Character lf;
                            nextPutAll: 'Se eliminaran las siguientes funciones: '].
    peliculaAeliminar verFunciones keysDo:
            [:key |
            funcionesPelicula := funcionesPelicula , (peliculaAeliminar verFunciones at: key) printString , '.'].
    confirmado := MessageBox
                confirm: '¿Seguro desea eliminar la Pelicula?
' , peliculaAeliminar verTitulo , ''
                            , peliculaAeliminar verFormato , funcionesPelicula.
    confirmado
        ifTrue:
            [model eliminarPelicula: peliculaAeliminar.
            self actualizarPeliculas]
```

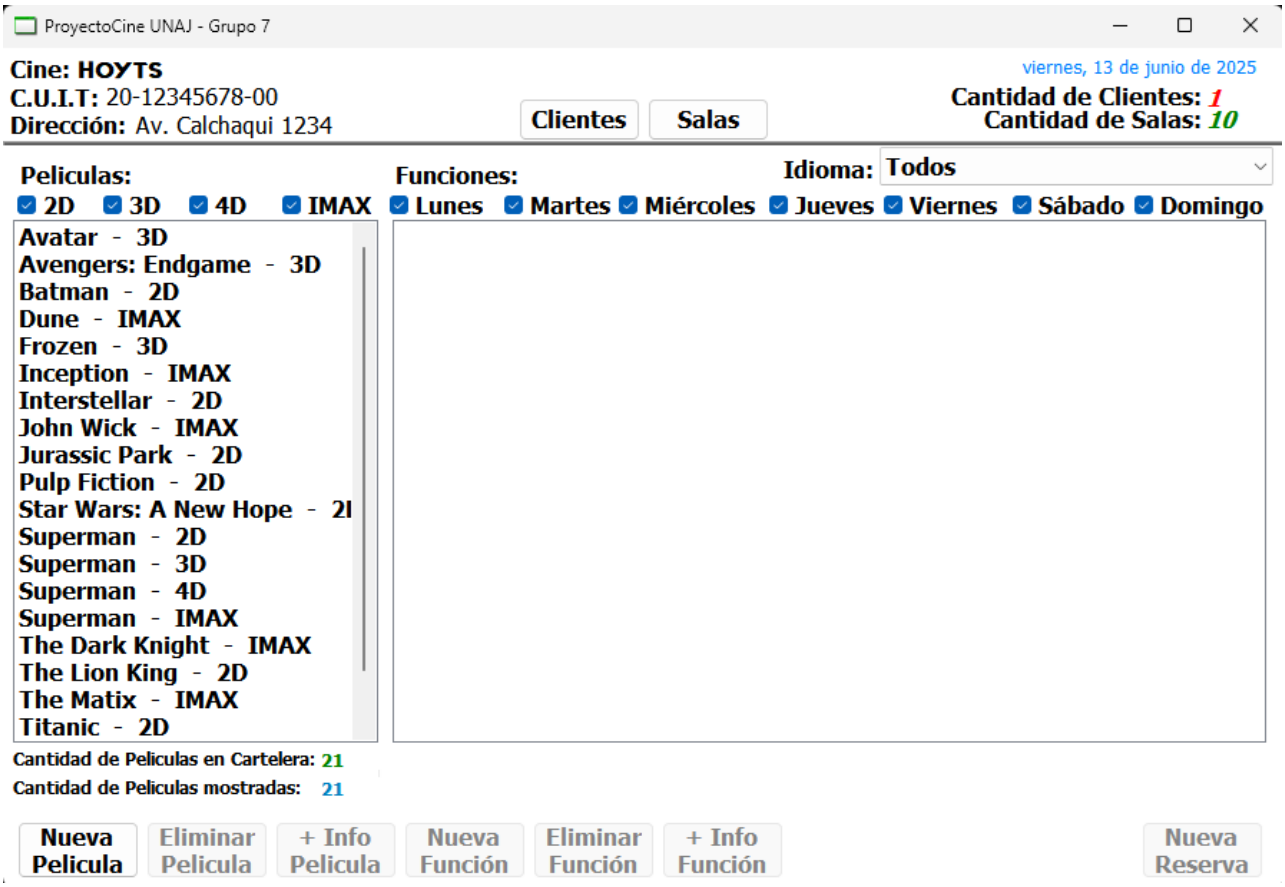
Clase CineNuevaReserva:

```
dibujarAsientosDesdeX: xPos desdeY: yPos
    "Asientos de la sala"

    | tpx tpy bp asientosDisp asientosRes cantFilas cantColumnas |
    asientosDisp := ownerView model verAsientosDisponibles: self model idFuncionPelicula
                    sala: (ownerView model buscarFuncionPelicula: self model idFuncionPelicula) verNombreSalaAsignada.
    asientosRes := ownerView model verAsientosReservados: self model idFuncionPelicula
                    sala: (ownerView model buscarFuncionPelicula: self model idFuncionPelicula) verNombreSalaAsignada.
    "El valor maximo de columna y fila"
    cantFilas := asientosDisp , asientosRes inject: (asientosDisp first at: 1)
                    into: [:max :subCollection | (subCollection at: 1) > max ifTrue: [subCollection at: 1] ifFalse: [max]].
    cantColumnas := asientosDisp , asientosRes inject: (asientosDisp first at: 2)
                    into: [:max :subCollection | (subCollection at: 2) > max ifTrue: [subCollection at: 2] ifFalse: [max]].

    "Dibujar Los numeros que representan el numero de Fila y Columna"
    1 to: cantColumnas
        do:
            [:x |
            1 to: cantFilas
                do:
                    [:y |
                    y = 1
                        ifTrue:
                            [tpx := TextPresenter createIn: self.
                            tpx extent: 30 @ 30.
                            tpx position: (x * 30 + xPos) @ yPos.
                            tpx view text: x printString.
                            tpx view isReadOnly: true].
                    x = cantFilas
                        ifTrue:
                            [tpy := TextPresenter createIn: self.
                            tpy extent: 30 @ 30.
                            tpy position: xPos @ (y * 30 + yPos).
                            tpy view text: y printString.
                            tpy view isReadOnly: true]]].
    "Dibujar los Asientos disponibles"
    1 to: asientosDisp size
        do:
            [:i |
            "Instancia un asientoReservaModel que representa el asiento seleccionado"
            | asi |
            asi := AsientoReservaModel new.
            asi fila: ((asientosDisp at: i) at: 1).
            asi columna: ((asientosDisp at: i) at: 2).
            asi estado: ((asientosDisp at: i) at: 3).
            "Agrega el asiento a la lista de asientos que luego se va a utilizar para la reserva si estado = true"
            self model asientos add: asi.
            "crea el checkbox que se utiliza para seleccionar el asiento"
            bp := BooleanPresenter createIn: self.
            bp extent: 30 @ 30.
            bp position: (((asientosDisp at: i) at: 2) * 30 + xPos) @ (((asientosDisp at: i) at: 1) * 30 + yPos).
            bp view isPushLike: true.
            bp view
                name: ((asientosDisp at: i) at: 1) printString , '-' , ((asientosDisp at: i) at: 2) printString.
            bp view value: asi estado.
            bp view isEnabled: true.
            bp view hasClientEdge: false.
            bp view hasBorder: false.
            "Vincula el model con el checkbox de selexion de asiento"
            bp model: ((self model asientos at: i) aspectValue: #estado).
            asientosPresenter add: bp].
    "Dibujar los asientos reservados"
    1 to: asientosRes size
        do:
            [:i |
            bp := BooleanPresenter createIn: self.
            bp extent: 30 @ 30.
            bp position: (((asientosRes at: i) at: 2) * 30 + xPos) @ (((asientosRes at: i) at: 1) * 30 + yPos).
            bp view isPushLike: true.
            bp view value: ((asientosRes at: i) at: 3).
            bp view isEnabled: false.
            bp view hasClientEdge: true.
            bp view hasBorder: true]
```

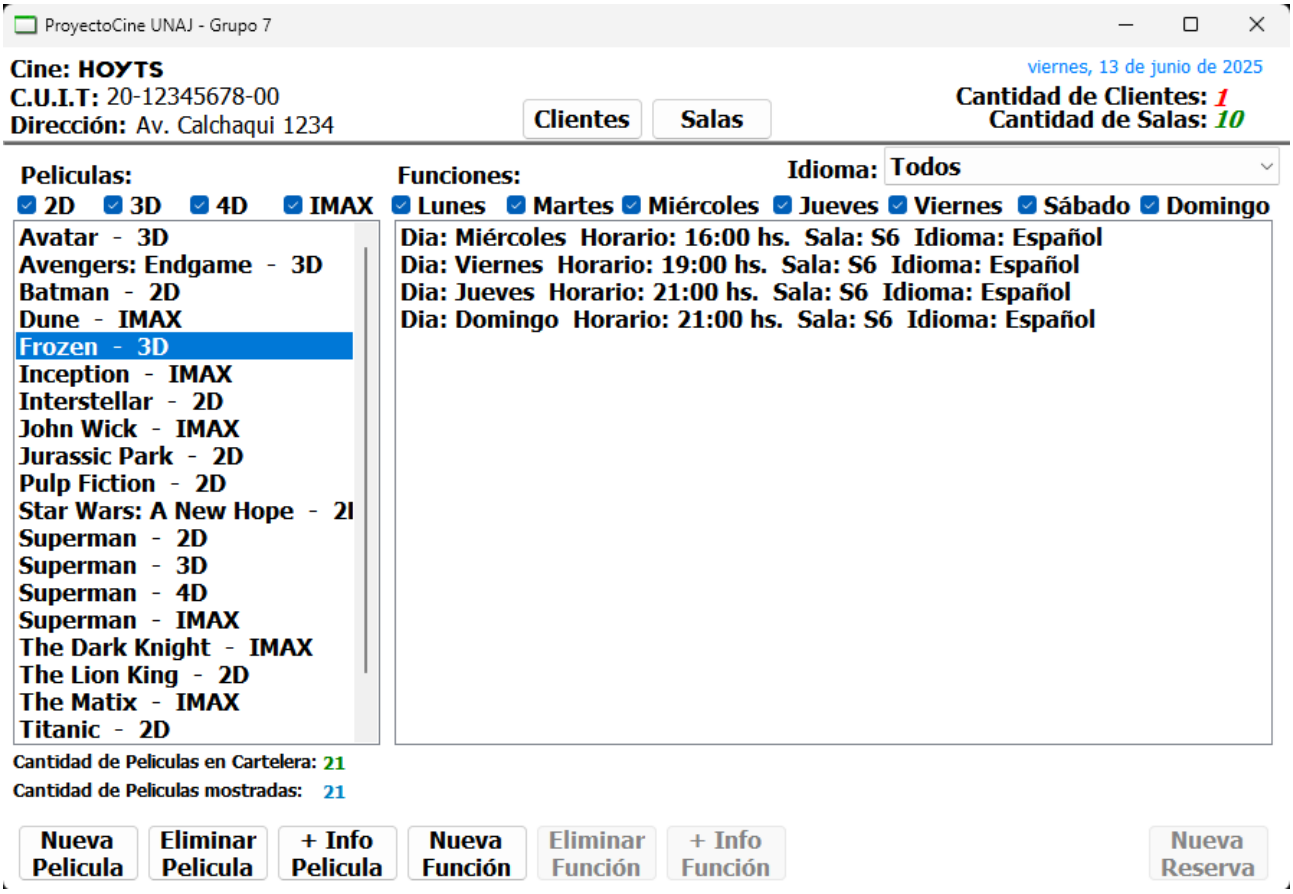
Captura de las Vistas de la aplicación:

Ejemplo de Instanciación inicial de la aplicación:

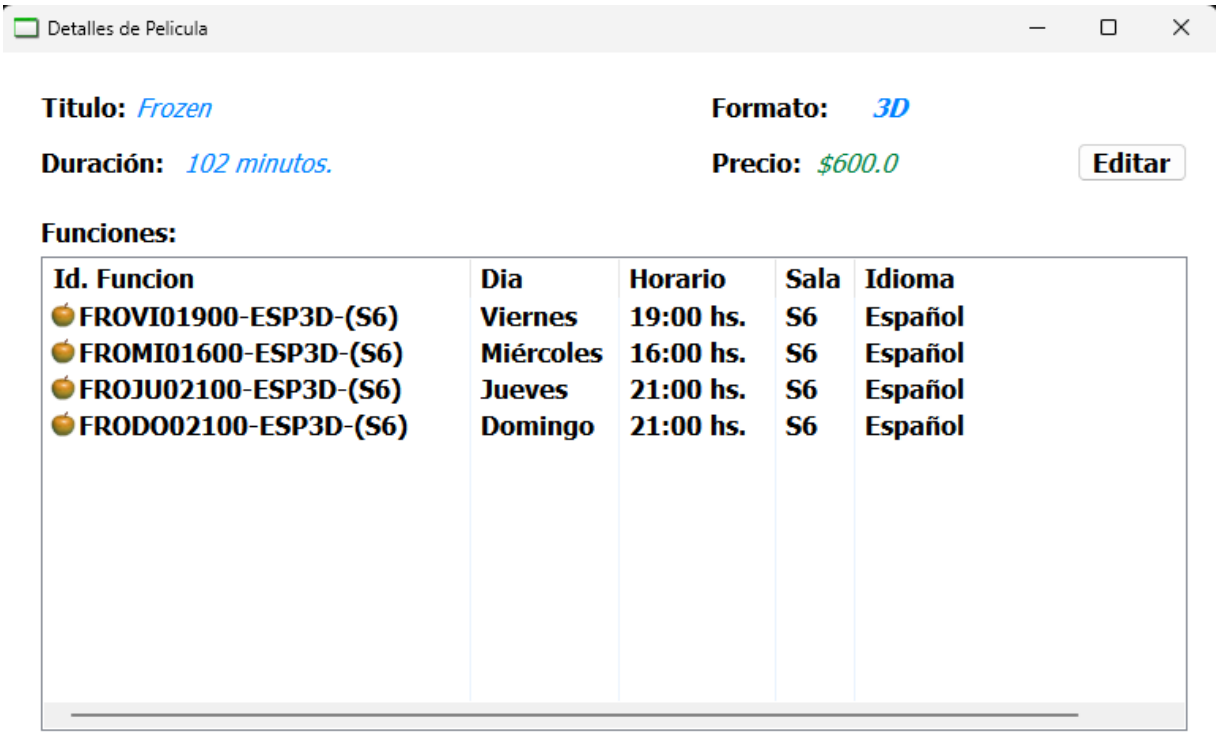
Ventana principal:



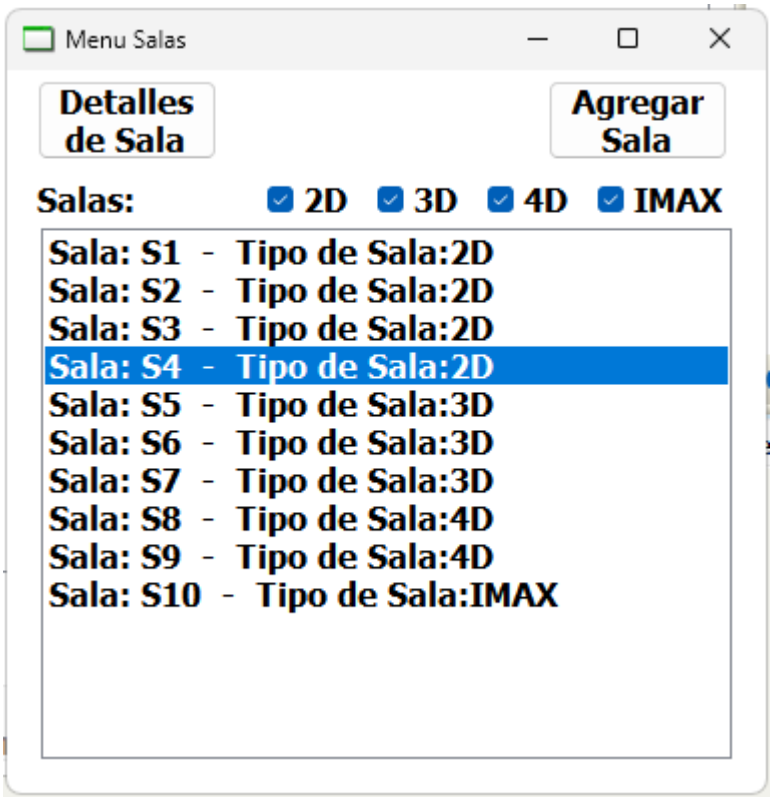
Visualización de las funciones al seleccionar una película:



Visualización Detallada de la información de una Pelicula:

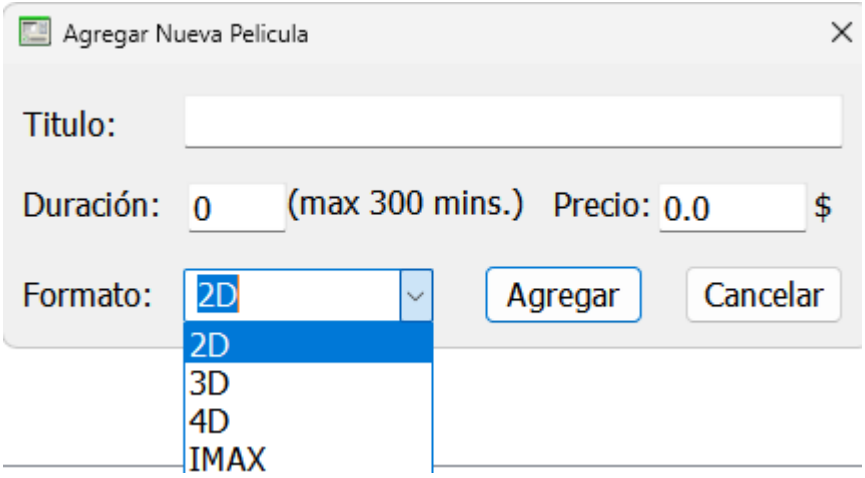


Menú de Salas:

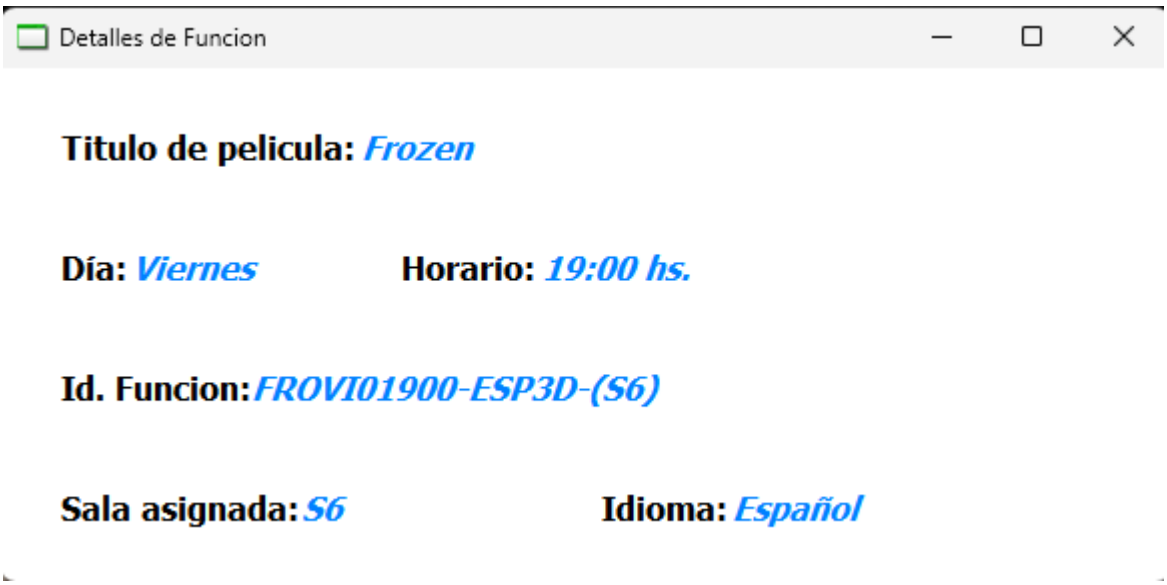


Visualización de información detallada de una Sala:

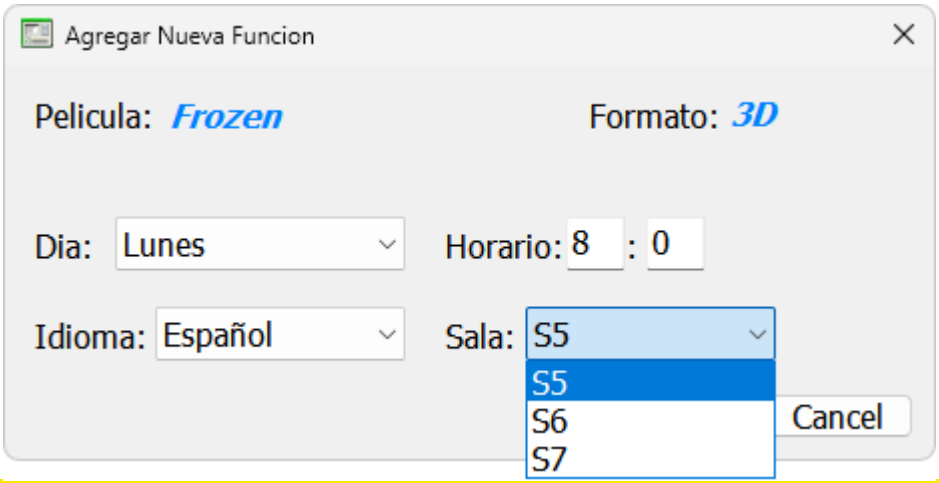
Ventana de formulario para agregar una nueva Pelicula:



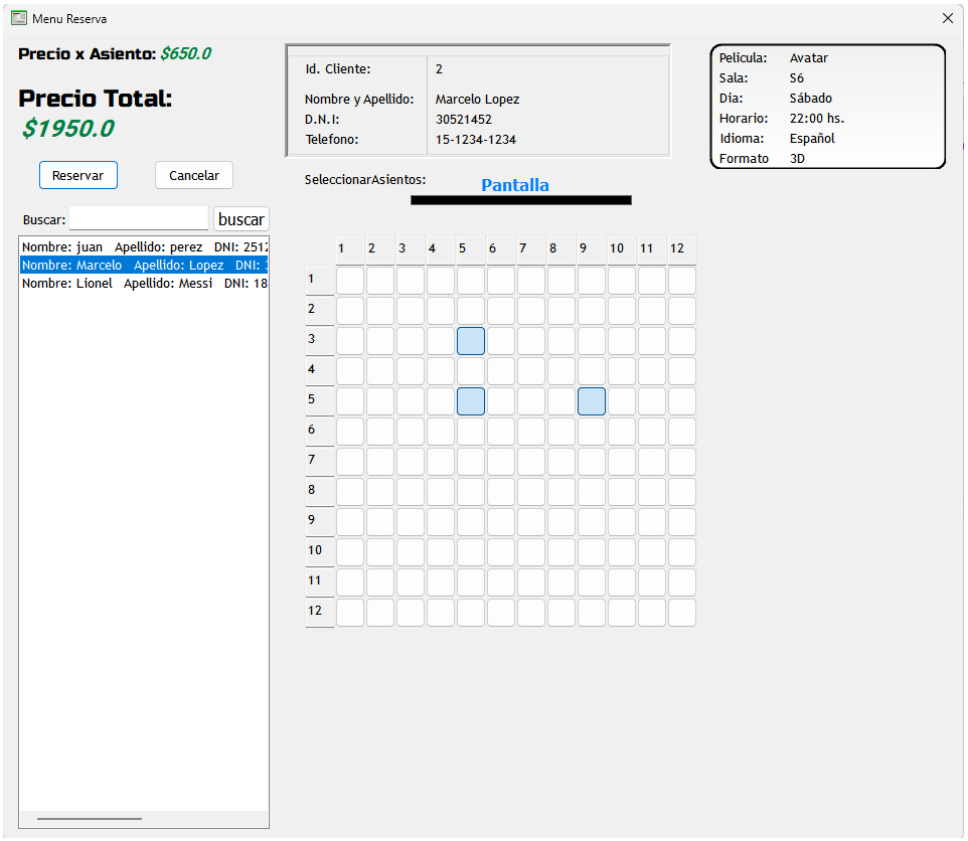
Visualización de información detallada de una función:



Ventana de formulario para agregar una nueva Función (mostrando únicamente Las salas disponibles para ese formato):



Menú de Reserva (con todos los asientos disponibles):



Menú de Reserva (sin haber elegido cliente aun y cuando ya existían asientos reservados):

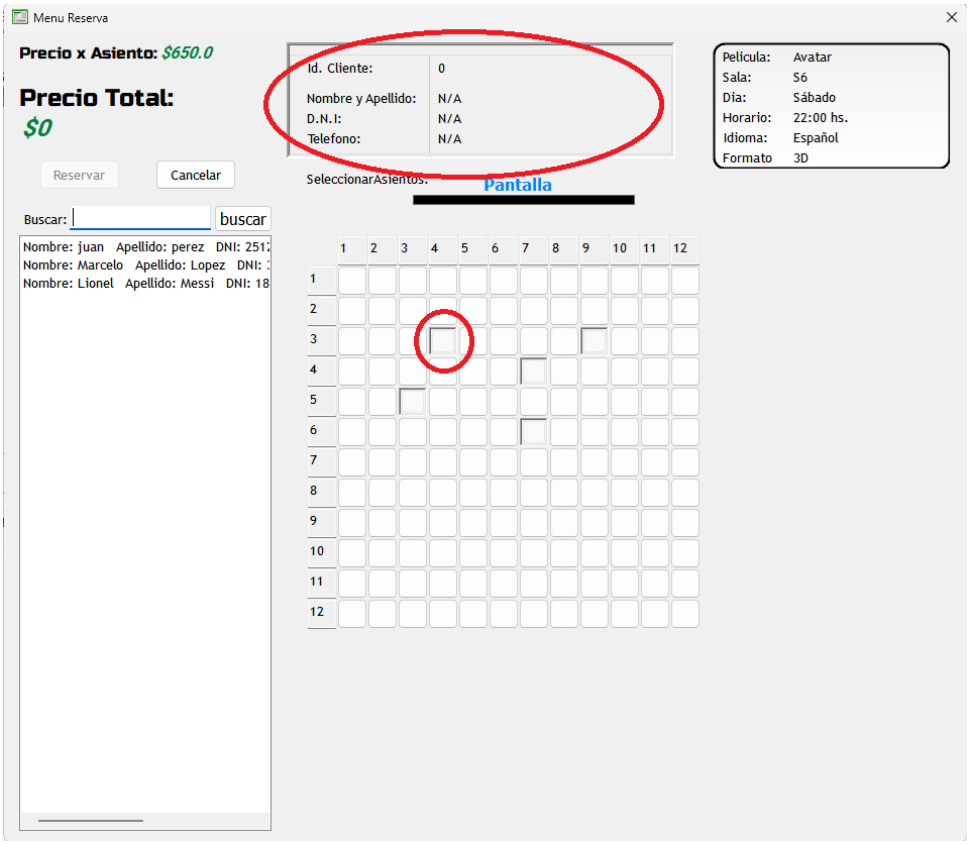


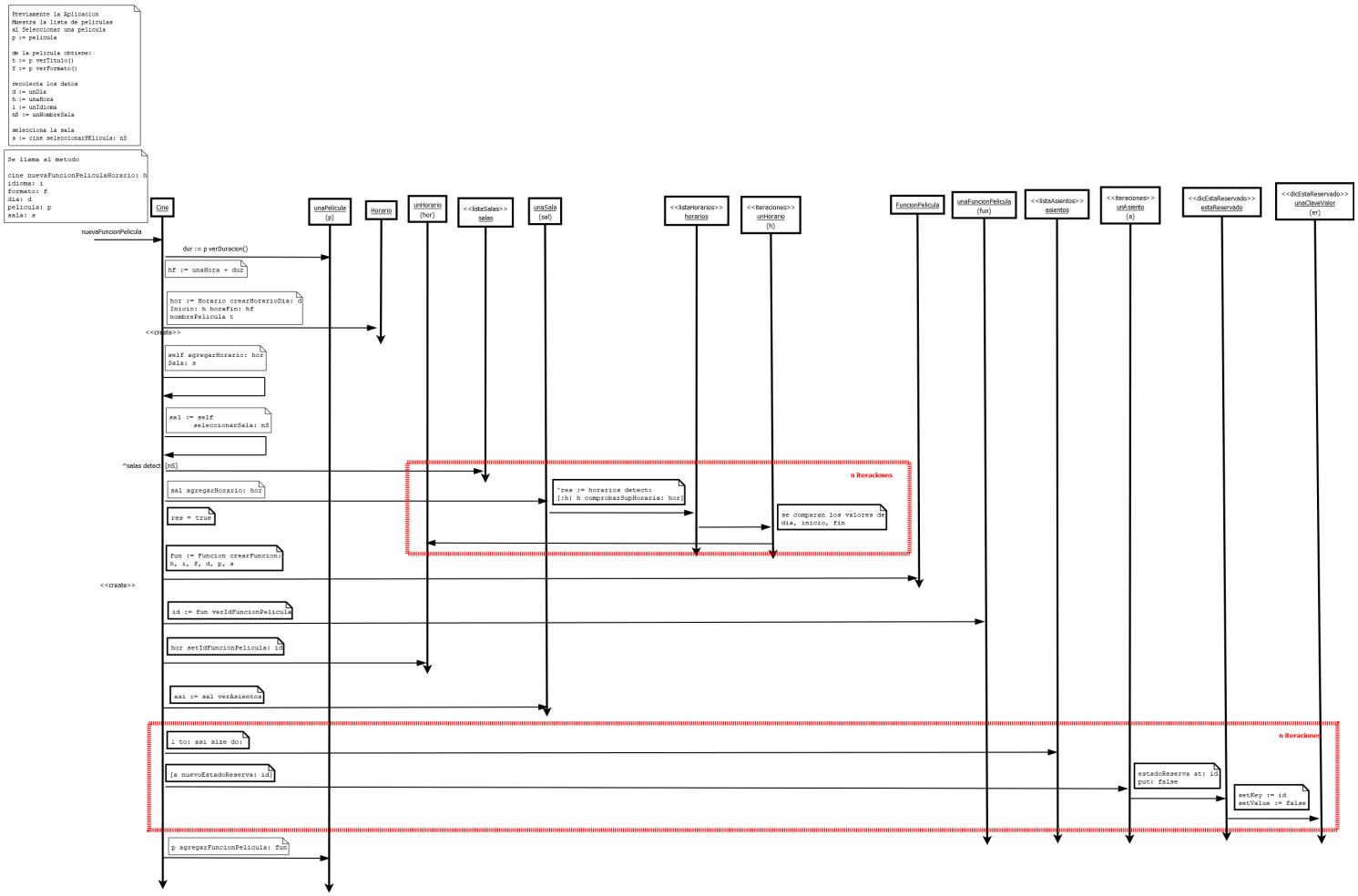
Diagrama de Secuencia – Registro de Función y Asignación de Asientos

El siguiente diagrama de secuencia representa el proceso de creación de una función de película en el cine, incluyendo la selección de película, sala, horario y asignación de asientos.

Este escenario refleja la interacción entre las clases principales del sistema, como Cine, Pelicula, Sala, FuncionPelicula, Horario y Asiento, demostrando cómo el sistema:

- Verifica la existencia de la película y la sala deseadas.
- Crea una nueva instancia de función, asignando horario y formato.
- Genera los objetos necesarios para representar la disponibilidad de los asientos durante esa función.
- Usa estructuras como colecciones (OrderedCollection, Dictionary) y métodos clave (select, detect, add:) para gestionar los datos.

Este diagrama permite visualizar de forma clara y detallada el flujo de mensajes entre objetos, asegurando que se respete la lógica del negocio y se mantenga la integridad del sistema durante la programación de funciones y la asignación de recursos.



Metodología de trabajo

El equipo implemento **una metodología híbrida**, combinando prácticas de **Extreme Programming (XP)**, **Scrum**, **Kanban** y **Scrumban**, adaptada a la disponibilidad de los integrantes, a las capacidades de cada integrante y la complejidad del código. Debido a la imposibilidad de coordinar horarios fijos, se utilizó **Nifty** para la organización del proyecto, estructurando un flujo de trabajo basado en **sprints** de tareas.

Etapas del desarrollo:

1. Planificación:
   - Definición del modelo de negocio y las funcionalidades clave.
   - Organización del flujo de trabajo y establecimiento de prioridades.
2. Diseño:
   - Desarrollo colaborativo de un **diagrama UML**, utilizando **Canva** como herramienta de trabajo.
   - Creación de una pseudo documentación con explicaciones breves y precisas de cada clase y método.
3. Implementación:
   - Asignación de tareas en **Nifty,** donde cada tarea representaba una clase contenía subtareas para definir la funcionalidad concreta.
   - Uso de estados de tarea representando el estado actual de cada tarea.
   - Compartición de código en archivos '**.cls**' (para clases individuales) o '**.pac**' (para múltiples clases).

Enfoques aplicados:

1. Tableros Kanban:
   - Se estructuraron tareas en **Nifty**, donde cada tarea representaba una clase, y cada subtarea definía una funcionalidad específica.
   - Los estados de las tareas incluían "*To Do*", "*En Proceso*", "*A revisar*", "*En Revisión*", y "*Finalizada*".
2. Scrum y Scumban:
   - Se realizo una planificación inicial para definir el modelo de negocio y las funcionalidades clave.
   - Se estableció una estructura de **sprints** flexibles, manteniendo un desarrollo continuo, basado en disponibilidad.
3. Extreme Programming (XP):
   - En tareas de mayor complejidad, se implementó **programación en parejas**, utilizando **Discord** para compartir pantalla, analizar código y depurar errores.
   - Se aplico **factorización**, refactorizando código para mejorar la legibilidad y hacer el código lo más simple posible.
4. Integración continua:
   - Cada nueva funcionalidad desarrollada era testeada y agregada inmediatamente, asegurando la estabilidad del software.

5. Testing y validación:
   - Se utilizo **Workspace** de **Dolphin Smaltallk** para crear instancias y probar métodos, verificando su funcionalidad esperada.
   - Se inspeccionaron objetos en cada parte testeada, asegurando que los datos y la estructura de los objetos fueran correctos antes de integrarlos al sistema.
   - En las etapas finales, se validó la correcta funcionalidad del sistema gráfico, verificando que la integración entre la lógica interna y la interfaz visual fuera fluida y sin errores.

Roles y colaboración

Aunque no hubo roles fijos, un integrante asumió una función de coordinador del equipo, organizando tareas y documentando las clases en base al UML, Sin embargo, la toma de decisiones se mantuvo en un único nivel de jerarquía, garantizando un trabajo en equipo equitativo y participativo.

Frameworks orientados a objetos utilizados y sugeridos

Para el desarrollo del proyecto se utilizó el entorno de desarrollo **Dolphin Smalltalk Professional 7.0.5**, un framework orientado a objetos puro que permite trabajar con **clases**, **instancias** y **mensajes** en un entorno visual enfocado en el paradigma de objetos.

Dolphin facilita la creación y gestión de clases, así como la interacción entre ellas, lo que lo hace ideal para proyectos **académicos** centrados en el diseño orientado a objetos.

Frameworks utilizados

- ✓ Dolphin Smalltalk
  - Proporciona un entorno de desarrollo grafico basado en el paradigma **MVP** (**Model-View-Presenter**).
  - Permite la creación de **interfaces ricas** y estructuradas, facilitando la interacción con el usuario.
  - Ofrece herramientas avanzadas para la manipulación de objetos y depuración en tiempo real.

Frameworks sugeridos para futuras expansiones

Pharo

- Un entorno moderno de Smalltalk con soporte activo y herramientas avanzadas.
- Facilita la creación de aplicaciones web, lo que podría ser útil para extender la aplicación del cine a una versión online.
- Incluye un ecosistema de paquetes que permite integración con base de datos y sistemas distribuidos.

Seaside

- Framework para aplicaciones web orientadas a objetos en **Smalltalk**.
- Permitiría transformar la aplicación actual en una **plataforma online**, donde los clientes puedan reservar funciones de manera remota.
- Implementa gestión de estado sin necesidad de uso intensivo de base de datos.

Magritte

- Sistema de meta-descripción en **Smalltalk**, útil para mejorar el modelado de datos dentro del cine.
- Facilita la creación de formularios dinámicos, lo que podría ser útil en la **gestión de clientes y empleados** en futuras expansiones.

Roassal

- Framework de visualización de datos en **Smalltalk**, permitiendo generar gráficos y diagramas sobre el estado del cine.
- Útil para representar métricas como **reservas**, **ocupación de salas** y **estadísticas de películas más vistas**.

Alternativas en otros lenguajes orientados a objetos

Si se planteara realizar este mismo sistema en otro lenguaje orientado a objetos, una opción adecuada seria **.Net con C# en el entorno de desarrollo Visual Studio**. Esta combinación ofrece:

- **Estructura robusta de clases** con soporte para patrones de diseño.
- **Herramientas de depuración avanzadas**, facilitando la resolución de errores.
- **Gran comunidad de desarrollo**, lo que permite acceder a documentación y recursos constantemente actualizados.

Además, **.NET** cuenta con frameworks como:

- **Windows Forms** y **WPF**, ideales para aplicaciones de escritorio con interfaces dinámicas.
- **ASP.NET MVC**, permitiendo escalar el proyecto a una aplicación web con mayor funcionalidad y una presentación más moderna.

Impacto de los frameworks en el proyecto

- **Expansión a una versión web** con **Seaside** y **Pharo**, mejorando accesibilidad.
- **Mejora del análisis de datos** con **Roassal**, permitiendo visualizar información comercial del cine.
- **Optimización de formularios y gestión** de clientes con **Maggrite**, facilitando futuras extensiones.
- **Migración a entornos más comerciales** mediante **.NET y C#**, ofreciendo mayor compatibilidad y escalabilidad.

Casos de uso de Patrones en el contexto del problema

Patrones de diseño aplicados:

Uso del Patrón Adapter en CineModel:

El **propósito del adapter** en **CineModel** es traducir y adaptar los datos de **Cine** para que sean utilizados en la interfaz grafica de **MVP**. Esto permite que **la interfaz no dependa directamente de la estructura interna** de **Cine**, manteniendo un nivel de abstracción entre la vista y la lógica.

Ejemplo de adaptación en CineModel

```
verFunciones: unaPelicula
    | funciones dias |
    dias := Dictionary new.
    dias at: 'Lunes' put: 1.
    dias at: 'Martes' put: 2.
    dias at: 'Miércoles' put: 3.
    dias at: 'Jueves' put: 4.
    dias at: 'Viernes' put: 5.
    dias at: 'Sábado' put: 6.
    dias at: 'Domingo' put: 7.
    funciones := (cine verFuncionesDePelicula: unaPelicula) asSortedCollection:
                        [:x :y |
                        (x mayorTitulo: y) or: [(x mayorDia: y) or: [(x mayorHorario: y) or: [x mayorSala: y]]]].
    ^funciones
```

Como funciona:

- **Cine verFuncionesDePelicula: unaPelicula** obtiene los datos directamente desde **Cine**.
- **CineModel adapta** los datos aplicando ordenamiento con **asSortedCollection**, asegurando que la interfaz los muestre correctamente.

Otro ejemplo – Adaptación de ver únicamente el nombre de las salas en forma de lista

```
verNombreSalasFormato: unFormato
    | nombreSalas |
    nombreSalas := cine verSalas select: [:sala | sala verTipo = unFormato].
    ^nombreSalas collect: [:sala | sala verNombreSala]
```

Como funciona:

- **Cine verNombreSalasFormato: unFormato** obtiene los datos directamente desde **Cine**.
- **CineModel adapta** los datos aplicando un filtrado de las salas obteniendo únicamente las que tienen el mismo formato que **unFormato** utilizando **select**. Y luego transformando esa lista utilizando **collect** para obtener únicamente el nombre de las salas.

Impacto del Adapter en la arquitectura

- ✓ **Separa la lógica interna del cien de la interfaz gráfica**, permitiendo cambios en **Cine** sin afectar la presentación.
- ✓ **Optimización de la manipulación de datos**, asegurando que la interfaz solo reciba la información que necesita.
- ✓ **Facilita futuras expansiones**, permitiendo agregar más adaptaciones sin alterar **Cine**.
- ✓ **Evita modificaciones en Cine**, lo que permite que **Cine** pueda ser utilizado en otros proyectos, incluso aquellos que no implementan interfaces gráficas.

Uso del Patrón Observer en CineModel

En **Dolphin Smalltalk**, los eventos observables se declaran en **publishedEventsOfInstances**, asegurando que los cambios en **CineModel** sean detectados por **CineShell** y otros componentes gráficos.

Ejemplo de declaración de eventos en CineModel

```
publishedEventsOfInstances
    ^(super publishedEventsOfInstances)
        add: #peliculasChanged;
        add: #salasChanged;
        add: #clientesChanged;
        add: #funcionesChanged;
        add: #reservasChanged;
        yourself
```

- Aquí se **declaran los eventos** que serán observados por la interfaz.
- Cuando **una película**, **sala** o **función cambia**, la UI se actualiza automáticamente.

Ejemplo de actualización al modificar el modelo

```
eliminarPelicula: unaPelicula
    cine eliminarPelicula: unaPelicula.
    self actualizarCantidadPeliculas
```

```
actualizarCantidadPeliculas
    cantidadPeliculas := cine verPeliculas size.
    self trigger: #peliculasChanged
```

Cuando una pelicula se elimina se llama a la funcion **actualizarCantidadPeliculas** donde:

Se llama **trigger: #peliculasChanged**, notificando a todos los observadores que las peliculas han cambiado.

La interfaz detecta el evento y **se actualiza automáticamente** sin intervención manual.

Impacto del Observer en la aplicación

- ✓ Sincronizacion automatica entre CineModel y la UI, evitando llamadas innecesarias.
- ✓ Facilitamantenimiento, ya que los cambios en el modelo se reflejan instantaneamente en la interfaz.
- ✓ Mejora rendimiento, reduciendo verigicaciones manuales sobre el estadod el cine.

Patrones sujeridos:

Patrón Singleton – Gestión única del Cine

El **patrón Singleton** se podria utilizar en la clase **Cine**, asegurando que **exista una unica instancia** del cine en ejecucion. Este enfoque evita la creacion de multiples objetos del mismo cine y garantiza que **toda la administracion se realice desde una unica fuente de datos.**

Patrón Factory Method – Creación de Reservas, Asientos, Peliculas, Funciones, Salas y Clientes

El **patrón Factory Method** se podria utilizar para **crear objetos de manera encapsulada**, evitando la creacion directa de instancias y delegando la logica de inicializacion a metodos especificos.

Patrón Proxy – Gestión de peliculas, clientes

El patrón Proxy se podria utilizar en la administracion de la cartelera, implementando un proxy de carga diferida, permitiendo que los datos de las peliculas se cargen cuando realmente sean necesarios.

En los clientes en lugar de permitir acceso directo a la informacion completa de Cliente, podriamos implementar un Proxy de Cliente, asegurando que solo ciertos datos sean accesibles de inmediato y otros sean cargados bajo demanda.

Repositorios para desarrollo colaborativo

Para el desarrollo de software colaborativo, es fundamental contar con repositorios que permitan **versionado**, **integración continua y trabajo en equipo eficiente**. Se exploraron varias opciones, cada una con sus ventajas según el entorno de desarrollo y necesidades del equipo.

Repositorios utilizados y recomendados

- ✓ **Nifty (gestión de tareas y archivos compartidos)**
  - Se utilizó para organizar el proyecto en **sprints y tableros Kanban**.
  - Permitió la **gestión de archivos**, donde el código se compartía en .cls para clases individuales o .pac para paquetes completos.
  - Facilitó la **colaboracion en equipo**, asignando tareas con estados (To Do, En Proceso, En Revisión, etc.). Y la **visualización** de en que estaba trabajando cada integrante.
- ○ **GitHub (control de versiones y desarrollo colaborativo en Smalltalk)**
  - Ideal para administrar código fuente y facilitar trabajo en equipo mediante **git.**
  - Compatible con **Squeak Smalltalk, Pharo** y otras variantes, asegurando facil integración con proyectos orientados a objetos.
  - Proporciona **herramientas de CI/CD**, permitiendo pruebas automáticas en cada version.
- ○ **GitLab (alternativa robusta con gestion de CI/CD avanzada)**
  - Similar a GitHub, pero con mayor enfoque en **automatización y despliegues continuos**.
  - Soporta repositorios privados, asegurando la **seguridad del código compartido**.
  - Integración con herramientas de testing y cobertura de codigo.
- ○ **SqueakSource (repositorio especializado en Smalltalk)**
  - Diseñado especificamente para **gestionar paquetes y proyectos en Smalltalk**.
  - Permite compartir codigo de manera estructurada con la comunidad Smalltalk.
  - Util para mantener un historial de versiones de clases y metodos.

Como se aplicaron en el proyecto

Nifty se uso para **gestión y colaboración**, organizando tareas y permitiendo compartir fragmentos de código.

Impacto de los repositorios en el proyecto

- ✓ **Facilitan el desarrollo colaborativo**, evitando conflictos en el codigo.
- ✓ **Mejoran la trazabilidad**, permitiendo revisar el historial de cambios en cada módulo del cine.
- ✓ **Optimizacion del flujo de trabajo**, asegurando que nuevas características se integren progresivamente sin afectar versiones anteriores.

Conclusión

El desarrollo de este proyecto representó una experiencia sumamente enriquecedora, tanto desde el punto de vista técnico como académico. Implementar un sistema completo utilizando **Dolphin Smalltalk 7.0.5**, un entorno basado en un **framework orientado a objetos puro**, nos permitió comprender en profundidad los fundamentos del **paradigma orientado a objetos** y su aplicación práctica en un proyecto real.

A lo largo del desarrollo, se trabajó con conceptos clave como encapsulamiento, modularidad, colecciones, patrones de diseño y separación de responsabilidades, utilizando una arquitectura basada en **Model-View-Presenter (MVP)**. También se integraron prácticas ágiles para organizar el trabajo en equipo, lo cual favoreció la colaboración y el avance continuo.

El uso de Smalltalk como lenguaje nos llevó a entender cómo cada objeto interactúa con otros a través de mensajes, lo que reforzó nuestra capacidad de pensar y diseñar sistemas verdaderamente orientados a objetos.

En definitiva, este proyecto no solo nos permitió modelar de forma eficiente la lógica de un complejo de cines, sino también fortalecer nuestras competencias en programación, diseño de software, trabajo en equipo y buenas prácticas de desarrollo.